Python Lists, part 1

CMSC 104 Section 2
October 19, 2026

Administrative notes

Quiz 2 will be returned this evening

No classwork or assignments today!! Classwork 6 will be assigned on Wednesday

We're going to change the topics covered the next two weeks just slightly

Original schedule

10/19 Loops & Booleans
10/21 Strings
10/26 Strings
10/28 Data Collections
11/2 Data Collections

New schedule

10/19 Lists
10/21 Lists when used in functions; some strings
10/26 Strings and how they compare to lists
10/28 Dictionaries
11/2 Summary of strings, lists & dictionaries and how to handle large collections of data

What you should take away from today's lecture

Lists

- Multi-valued data type
  - Mutable; can change individual list values
- Adding values (append, insert)
- Removing values (remove, del)
- len
- Indexing
  - You always start counting at 0
  - You can use negative index values to start counting from the end instead of the beginning
- Often used with a different type of "for" loop that processes each list element exactly once
- List elements don't have to be the same type
- You can have lists of lists

How many values can a variable hold?

Up until now, every variable can hold exactly one value

- One integer, one string, one float, one boolean

But what if you want to have more than one value in a single variable?

- Why? Because of convenience.

Let's look at Project 2

Inconvenient, right? The notation is a pain in the neck?

```
def break_out_numbers(num):
        n6 = num % 10
        n5 = (num // 10) % 10
        n4 = (num // 100) % 10
        n3 = (num // 1000) % 10
        n2 = (num // 10000) % 10
        n1 = (num // 100000) % 10
        return n1, n2, n3, n4, n5, n6
```

Suppose we had a way to refer to all six digits at once

We do, and it’s called a “list” and it’s one of the fundamental “Pythonic” data structures

- “Pythonic” meaning that’s what makes Python unique among programming languages

If the variable “digits” is a list, then “digits” can hold as many different values as we want. Let’s revise the code from Project 2 to use a list, and then we’ll break down all the parts of it.

Alternative code, using lists

```
from random import randint

def break_out_numbers(number):
    """
    This function takes a six digit integer and breaks it
into a list six independent digits.
    The list is returned
    :param number: a six digit integer
    :return: digits: a list of six one-digit integers
    """
    digits = []
    print(number)
    for i in range(0, 6):
        digits.append((number//(10**(5-i)))%10)

    """
    There's another way to do this that might be simpler.
    We'll explain this either today or Wednesday

    for digit in str(number):
        digits.append(int(digit))
    """
    return digits
```

```
if __name__ == "__main__":
    n = randint(100000, 999999)
    d = break_out_numbers(n)
    print(d)
```

Creating a list

If you want to create an empty list:

```
l = [ ]   # note that these are square brackets, not parentheses or curly braces
```

If you want to create a list that already has values:

```
usm = [“UMBC”, “UMCP”, “UMES”, “Salisbury”, “Towson”]
```

If you want to convert existing values into a list:

If you want to convert something into a list, use the list constructor statement

```
letters = list(“UMBC”)
```

List indices

Each element in the list can be directly referred to by its index

- Indices ALWAYS start at 0, not at 1

```
countries = [“Canada”, “USA”, “Mexico”, “Jamaica”, “Panama”]
```

`countries[1]` is “USA”NOT “Canada”. “Canada” is `countries[0]`

The len function

len() is a built-in function that tells you how many entries are in a list.

NOTE: the last item in the list has index of len() -1.

From the previous slide:

```
countries = [“Canada”, “USA”, “Mexico”, “Jamaica”, “Panama”]
```

`len(countries)` returns the value 5. But if you look for `countries[5]` it’s an error; there is no such element in the list.

len(), continued

You can use formulas or variables to calculate the index; you can say

```
countries[len(countries)-1]
```

This is always the last element in the list.

So here's an opportunity for computer scientists to be lazy. We can drop that whole "len(countries)" bit and just say

```
Countries[-1]
```

That is, the LAST entry in a list ALWAYS has index -1. And we can count backward from there.

Adding one element to a list

- You can add one element to the end of a list using .append()
- You can add one element into a list at a specified index using .insert()
- *Strictly speaking, you can't add more than one element to a list at a time. You *can* concatenate two lists together; we'll get to that later. But you can only add one element at a time*

Removing one or more elements from a list

.remove() - removes a specific element, by value

- If the same value appears multiple times, only the first is removed.
- If the value does not appear in your list, it’s an error

.pop() - removes whatever value appears as the specified index

- If the index is invalid for the list, it’s an error

del removes a set of elements at one time

Lists are mutable

When you modify a list, it is modified in place. It stays in the same location(s) in memory; it is not moved around

This is different from int, string, float and boolean variables. Those are immutable. If you try to change the value they get moved around in memory.

This mutability is both and advantage and a disadvantage when coding.

- More about this is Wednesday's lecture

List slicing

You can access a sub-list - a subset of the original list - using a technique called *list slicing.*

You do that by specifying:

- The starting index; that is, the index of the first element in your slice
- The ending index. That is the index of the first element NOT in your slice. So that is, if you specify

  `countries[1,3]`

  You will get `countries[1]` and `countries[2]`. You wil NOT get `countries[3]`

  Yes, this can be confusing. And it's the cause of many many programming errors. But it's the same way that the "for i" loop works

About that new kind of “for” loop

If you have a list, AND

You want to do something with each element in the list, exactly one time, without skipping any elements, AND

You don’t want to modify the list in place, THEN

You can use what is called a “for each” loop. It’s simpler than either “for i” or “while” but it’s not very general; it only works in the situations described here.

Note: this is called a “for each” loop for historical reasons. In other language, the syntax requires the word “each.” “each” included in Python; the syntax is different. But we still call the structure a “for each” loop.

“For each” loop

```
counties = ["Allegany", "Anne Arundel","Baltimore", "Baltimore City", "Calvert",
"Carroll", "Caroline", "Cecil", "Charles", "Dorchester", "Frederick", "Garrett",
"Harford", "Howard", "Kent", "Montgomery", "Prince George's", "Queen Anne's",
"Somerset", "St. Mary's","Talbot", "Washington", "Wicomico", "Worcester"]

for county in counties:

    print(county)
```

We want to do the same thing - print the value - for each item in the list, exactly one time and skipping no list elements. So we can use the simpler “for each” loop to do this.

This is another example of “syntactic sugar.” There’s no reason we can’t do this same thing with a “while” loop; it works perfectly well. But the “for each” is syntactically simpler, so sure, let’s do that.

lists.py

There is a file in the code_samples repo on the glass GitHub repo called “lists.py.” It provides samples of everything we’ve done today.